

Why Airflow is a Great Choice for This Task

Airflow is an excellent choice for automated, scheduled tasks like this. Here are a few reasons why it's well-suited for your needs:

- **Scheduled and Recurring Runs**: Airflow is specifically designed to handle workflows that need to run at regular intervals. You can easily manage schedules for daily, weekly, or any custom frequency.
- **DAG-Based Workflow Control**: Airflow allows you to define individual steps in the workflow as separate tasks (e.g., data loading, model training, result evaluation), making the entire process more transparent and easier to manage.
- **Conditional Decision-Making**: Since you want to check if the new model outperforms the previous one, Airflow makes it straightforward to set up conditional logic for these kinds of decisions.
- **Monitoring and Error Handling**: Airflow provides built-in monitoring tools, so if a task fails, you can receive notifications, and the system can automatically retry the run.
- **MLflow Integration**: Airflow integrates well with MLflow, allowing you to control model versions, register new models, and manage version transitions as needed.

Setting Up and Configuring Airflow Locally

Here's how you can install and configure Airflow on your local machine:

1. Installing Airflow Locally

- Open a terminal and run the following commands to set up Airflow:

```
```bash
Create a new directory for Airflow
mkdir ~/airflow
cd ~/airflow

Set the Airflow Home environment variable
export AIRFLOW_HOME=~/airflow

Install Airflow
pip install "apache-airflow==2.6.3" --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-
2.6.3/constraints-3.8.txt"
```
```

- This will download and install the necessary packages for the specified Airflow version.

2. Initializing Default Settings

- Initialize Airflow's default database and tables by running:

```
```bash
airflow db init
```
```

3. Creating an Admin User

- Create an admin user to access the Airflow web interface:

```
```bash
airflow users create \
 --username admin \
 --firstname Admin \
 --lastname User \
 --role Admin \
 --email admin@example.com
```
```

4. Starting Airflow

- Start the Airflow web server:

```
```bash
airflow webserver --port 8080
```
```

- In a new terminal window, start the Airflow scheduler:

```
```bash
airflow scheduler
```
```

- Once running, Airflow will be accessible in your browser at [\[http://localhost:8080\]](http://localhost:8080) (<http://localhost:8080>).

5. Adding a DAG for Model Training

- Create a new `.py` file for the DAG (e.g., `train_model_dag.py`) as described above, and save it in Airflow's `dags` folder (`~/airflow/dags/`).
- You should now see the new DAG in the Airflow interface, scheduled to run daily.

Customizing the Airflow DAGs Directory

By default, Airflow loads DAGs from its `dags` folder. However, you don't have to store your DAG files there; you can specify a custom directory in the Airflow configuration file if you prefer to keep them elsewhere.

Steps to Set a Custom DAGs Directory

1. Edit the Configuration File

- Open the Airflow configuration file (`airflow.cfg`), which is located in the main Airflow directory (`~/airflow` by default).
- Locate the following line in the configuration file:

```
```ini
dags_folder = /path/to/airflow/dags
```
```

– Change it to the directory where you want to store your DAG files. For example:

```
```ini
dags_folder = /my/custom/dags/folder
```
```

2. Restart Airflow

– After updating the `dags\_folder` path, restart both the Airflow web server and the scheduler to apply the changes:

```
```bash
airflow webserver --port 8080
airflow scheduler
```
```

After completing these steps, Airflow will load DAG files from the new directory you specified.